ASP-based Large Neighborhood Prioritized Search for Course Timetabling

Irumi Sugimori[1] Katsumi Inoue[2] Hidetomo Nabeshima[3]
Torsten Schaub[4] Takehide Soh[5] Naoyuki Tamura[5] Mutsunori Banbara[1]

[1]Nagoya University

[2]National Institute of Informatics

[3]University of Yamanashi

[4]Universität Potsdam

[5]Kobe University

LPNMR2024@Dallas
October 14th, 2024

Overview

- **Curriculum-based Course Timetabling** (CB-CTT) is one of the most studied educational timetabling problems. CB-CTT has been used in international timetabling competitions.
- **Large Neighborhood Prioritized Search** (LNPS; [Sugimori+,'24][1]) is a hybrid between systematic and stochastic local search for solving Combinatorial Optimization Problems (COPs).

Contributions

We present an approach to solve CB-CTT with LNPS based on ASP.

1. We develop domain-specific LNPS heuristics for CB-CTT solving.
2. The resulting *teaspoon-lnps* system demonstrates that LNPS can significantly enhance the performance of ASP for CB-CTT solving.

[1] Large Neighborhood Prioritized Search for Combinatorial Optimization with Answer Set Programming, KR2024

Timetabling

Timetabling is the task of assigning a set of entities (e.g., tasks, events, people) to the limited number of resources over time, subject to a given set of hard and soft constraints.

- The typical topics of this area include:

  Educational timetabling,
  Healthcare timetabling,
  Sports timetabling.
  Transport timetabling,
  Employee timetabling,

- International conferences and competitions have been held:
  - **PATAT**: International Series of Conferences on the Practice and Theory of Automated Timetabling ▸ Web
  - **ITC**: International Timetabling Competitions ▸ Web

Timetabling has received increasing attention from both researchers and practitioners.

Curriculum-based Course Timetabling (CB-CTT)

CB-CTT is defined as the task of assigning all lectures into a weekly timetable, subject to a given set of hard and soft constraints.

Hard constraints	**Soft constraints**	
H_1. Lectures	S_1. RoomCapacity	S_6. StudentMinMaxLoad
H_2. Conflicts	S_2. MinWorkingDays	S_7. TravelDistance
H_3. RoomOccupancy	S_3. IsolatedLectures	S_8. RoomSuitability
H_4. Availability	S_4. Windows	S_9. DoubleLectures
	S_5. RoomStability	

- Each lecture belonging to a **course** must take place in a **room** at a **period** on a **day**.
- The **hard constraints** must be strictly satisfied.
- The **soft constraints** are not necessarily satisfied but the sum of their violations should be minimal.

Formulations

- Soft constraints are different in each **formulation**.
- **UD2** was used in ITC-2007. **UD5** is the most difficult formulation.

Constraint	UD1	UD2	UD3	UD4	UD5
H_1. Lectures	H	H	H	H	H
H_2. Conflicts	H	H	H	H	H
H_3. RoomOccupancy	H	H	H	H	H
H_4. Availability	H	H	H	H	H
S_1. RoomCapacity	1	1	1	1	1
S_2. MinWorkingDays	5	5	-	1	5
S_3. IsolatedLectures	1	2	-	-	1
S_4. Windows	-	-	4	1	2
S_5. RoomStability	-	1	-	-	-
S_6. StudentMinMaxLoad	-	-	2	1	2
S_7. TravelDistance	-	-	-	-	2
S_8. RoomSuitability	-	-	3	H	-
S_9. DoubleLectures	-	-	-	1	-

teaspoon encoding [Banbara+,'13,'19]

teaspoon encoding is a collection of ASP encodings for CB-CTT solving.

Lessons learned from teaspoon encoding

A proper way of using different atoms depending on constraints can improve the efficiency of CB-CTT solving.

- The most direct encoding uses the predicate `assigned/4` only.
  - The atom `assigned(C,R,D,P)` represents that a lecture of a course `C` is assigned to a room `R` at a period `P` on a day `D`.
- The *teaspoon* encoding uses two different predicates `assigned/3` and `assigned/4` depending on constraints.
  - The atom `assigned(C,D,P)` drops the room information.

How does assigned/4 construct the solution?

The partial solution on the left forms the weekly timetables for the two curricula on the right.

partial answer set

```
assigned("TecCos", rB,0,1).
assigned("TecCos", rB,0,3).
assigned("TecCos", rB,1,3).
assigned("TecCos", rB,2,3).
assigned("TecCos", rB,4,3).
assigned("SceCosC",rB,3,0).
assigned("SceCosC",rB,2,2).
assigned("SceCosC",rB,4,2).
assigned("ArcTec", rB,3,1).
assigned("ArcTec", rB,0,2).
assigned("ArcTec", rB,1,2).
assigned("Geotec", rA,4,1).
assigned("Geotec", rA,0,2).
assigned("Geotec", rA,1,2).
assigned("Geotec", rA,2,2).
assigned("Geotec", rA,4,2).
```

Cur1

	Day0	Day1	Day2	Day3	Day4
0				SceCosC rB	
1	TecCos rB			ArcTec rB	
2	ArcTec rB	ArcTec rB	SceCosC rB		SceCosC rB
3	TecCos rB	TecCos rB	TecCos rB		TecCos rB

Cur2

	Day0	Day1	Day2	Day3	Day4
0					
1	TecCos rB				Geotec rA
2	Geotec rA	Geotec rA	Geotec rA		Geotec rA
3	TecCos rB	TecCos rB	TecCos rB		TecCos rB

Motivation

- ***teaspoon*** demonstrated that ASP can compete with state-of-the-art solving techniques on course timetabling, including
  - metaheuristic algorithms, integer programming, SAT/MaxSAT, etc.
- In fact, *teaspoon* managed to either improve or reproduce the best known bounds for 182 out of 305 combinations in total (all 61 instances in 5 formulations).

Motivation

- ***teaspoon*** demonstrated that ASP can compete with state-of-the-art solving techniques on course timetabling, including
  - metaheuristic algorithms, integer programming, SAT/MaxSAT, etc.
- In fact, *teaspoon* managed to either improve or reproduce the best known bounds for 182 out of 305 combinations in total (all 61 instances in 5 formulations).

Limitation of *teaspoon*

Especially for UD5, systematic search of ASP solvers quickly falls into saturated solutions for many instances.

Motivation

- ***teaspoon*** demonstrated that ASP can compete with state-of-the-art solving techniques on course timetabling, including
  - metaheuristic algorithms, integer programming, SAT/MaxSAT, etc.
- In fact, *teaspoon* managed to either improve or reproduce the best known bounds for 182 out of 305 combinations in total (all 61 instances in 5 formulations).

Limitation of *teaspoon*

Especially for UD5, systematic search of ASP solvers quickly falls into saturated solutions for many instances.

Idea of this research

We tackle this problem issue by taking advantage of **Large Neighborhood Prioritized Search** (LNPS).

Large Neighborhood Prioritized Search [Sugimori+,KR'24]

LNPS is a metaheuristic that starts with an initial solution and then iteratively tries to find better solutions by alternately destroying a current solution and reconstructing it with prioritized search.

- The prioritized search is a systematic search for which its branching heuristic can be configured or customized.
- The prioritized search can be implemented with heuristic-driven ASP solving (viz. `#heuristic` statement).

Algorithm 1 LNPS

Input: a feasible (current) solution x
Output: the best solution x^*

1: $x^* \leftarrow x$
2: **while** stop criterion is not met **do**
3: $\quad x^t \leftarrow \mathit{prioritized\text{-}search}(\mathit{destroy}(x))$
4: $\quad$ **if** $\mathit{accept}(x^t, x)$ **then**
5: $\quad\quad x \leftarrow x^t$
6: $\quad$ **end if**
7: $\quad$ **if** $c(x^t) < c(x^*)$ **then**
8: $\quad\quad x^* \leftarrow x^t$
9: $\quad$ **end if**
10: **end while**
11: **return** x^*

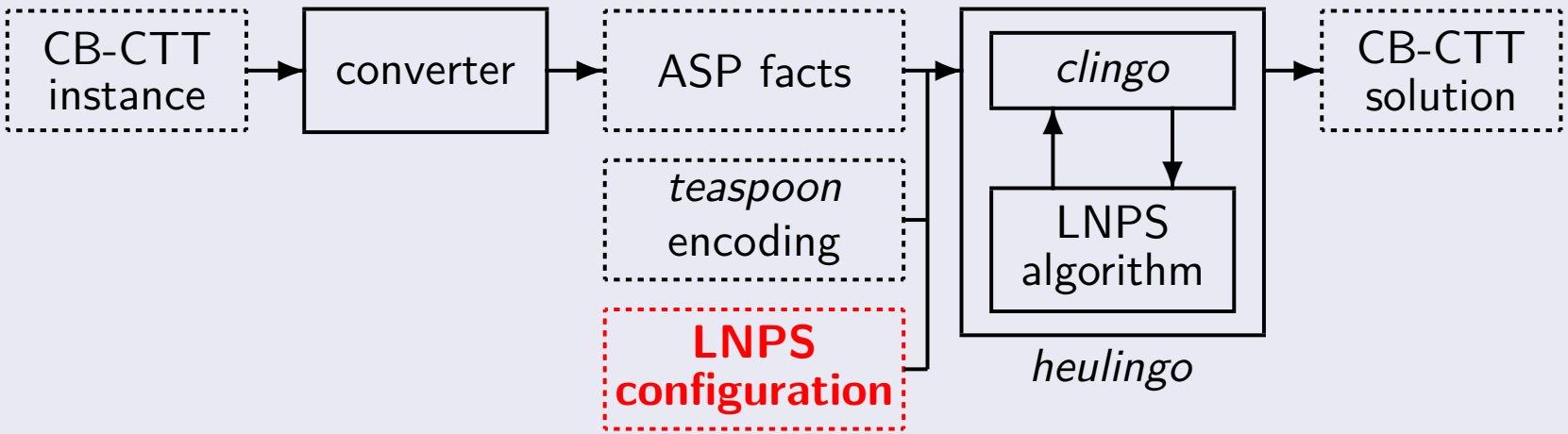
heulingo: an ASP-based implementation of LNPS

- **Variability**: *heulingo* provides a flexible search without strongly depending on the destroy operators compared to traditional LNS [Shaw,'98], since the undestroyed part is not fixed.
- **Optimality**: *heulingo* can guarantee the optimality of solutions.
- **Domain heuristics**: *heulingo* allows for easy incorporation of domain-specific heuristics in a declarative way.
- **Efficiency**: *heulingo* has demonstrated that LNPS can significantly enhance the solving performance of ASP on hard optimization problems, such as traveling salesperson, social golpher, sudoku generation, weighted strategic company and so on.

We focus on the development of domain-specific LNPS heuristics for efficient CB-CTT solving.

teaspoon-lnps approach (proposal)

We develop an approach to solve CB-CTT with ASP-based LNPS.



1. The resulting *teaspoon-lnps* solver accepts a CB-CTT instance and converts it into ASP facts.
2. In turn, these facts are combined with *teaspoon* encoding and an LNPS configuration, which are afterward solved by *heulingo*.

- LNPS configurations define the behavior of the LNPS heuristic, especially for the destroy and prioritized-search operators.

LNPS configurations

We develop five LNPS configurations for CB-CTT solving.

- **Random** is the simplest domain-independent heuristic that randomly destroys a current solution.
- **Day-Period** and **Day-Room** are domain-specific heuristics inspired by the traditional LNS heuristic for CB-CTT solving [Kiefer+,'17].
- Swap-Room and DP-Swap-Room are novel domain-specific configurations for CB-CTT solving, taking advantage of prioritized search.

Random N

Random N randomly destroys N% of a current solution and keeps the undestroyed part as much as possible in each iteration of LNPS.

```
1  #program config.
2  _lnps_project(assigned,4).
3  _lnps_destroy(assigned,4,15,p(n)).
4  _lnps_prioritize(assigned,4,1,true).
```

- The atom `_lnps_project(assigned,4)` means that the atoms of `assigned/4` belonging to an answer set are subject to LNPS.
- The atom `_lnps_destroy(assigned,4,15,p(n))` means that `n`% of a current solution characterized by `assigned/4` are destroyed.
  - The 3rd argument $15 = (1111)_2$ represents that all possible tuples (`C`,`R`,`D`,`P`) of `assigned(C,R,D,P)` are subject to destruction.

Random N

Random N randomly destroys N% of a current solution and keeps the undestroyed part as much as possible in each iteration of LNPS.

```
1  #program config.
2  _lnps_project(assigned,4).
3  _lnps_destroy(assigned,4,15,p(n)).
4  _lnps_prioritize(assigned,4,1,true).
```

- The atom `_lnps_prioritize(assigned,4,1,true)` means that the undestroyed part is kept as much as possible.
- Technically, this atom corresponds to *clingo*'s heuristic statement

  `#heuristic assigned(C,R,D,P):`*Body*`. [1,true]`.

Day-Period

Day-Period randomly selects a single day-period pair. In turn, it destroys parts of a current solution including the selected pair and keeps the undestroyed part as much as possible.

```
1  #program config.
2  _lnps_project(assigned,4).
3  _lnps_destroy(assigned,4,3,n(1)).
4  _lnps_prioritize(assigned,4,1,true).
```

- The difference from Random N is the fact in Line 3.
- The fact means that parts of a current solution characterized by `assigned/4` containing a randomly selected **day-period pair** are destroyed.
  - The 3rd argument $3 = (0011)_2$ represents that all possible pairs (`D`,`P`) of `assigned(C,R,D,P)` are subject to destruction.

Swap-Room *N*

- Swap-Room *N* is similar to Random *N*, but tries to keep course-day-period assignments as much as possible.
- This is designed to find better solutions by swapping rooms in the current solution.

```
1  #program config.
2  _lnps_project(assigned,4).
3  _lnps_destroy(assigned,4,15,p(n)).
4  _lnps_prioritize(assigned,4,1,true).
5  _lnps_project(assigned,3).
6  _lnps_destroy(assigned,3,7,p(0)).
7  _lnps_prioritize(assigned,3,1,true).
```

- The difference from Random *N* is the addition of Lines 5–7.
- The additional facts mean that the course-day-period assignments represented by `assigned/3` are kept as much as possible.

DP-Swap-Room N

- DP-Swap-Room N is similar to Swap-Room N, but randomly selects N day-period pairs and destroys all atoms including any selected pair.
- This is designed to find better solutions by swapping rooms at N day-period pairs in the current solution.

```
1  #program config.
2  _lnps_project(assigned,4).
3  _lnps_destroy(assigned,4,3,n(n)).
4  _lnps_prioritize(assigned,4,1,true).
5  _lnps_project(assigned,3).
6  _lnps_destroy(assigned,3,7,p(0)).
7  _lnps_prioritize(assigned,3,1,true).
```

- The difference from Swap-Room N is the fact in Line 3.
- The fact means that parts of a current solution containing any one of randomly selected `n` day-period pairs are destroyed.

Experiments

We carry out experiments to evaluate our *teaspoon-lnps* approach.

- We use all 61 instances of standard CB-CTT benchmark set within the most difficult **UD5 formulation**.
- ASP encoding: *teaspoon* encoding [Banbara+,'13,'19]
- We compare
  - *clingo*-5.6.2 with the two best options [Banbara+,'19].
  - ***teaspoon-lnps*** with the five configurations:
    1. Random N with 3 different percentages $N \in \{2, 4, 6\}$
    2. Day-Period
    3. Day-Room
    4. Swap-Room N with 3 different percentages $N \in \{8, 10, 12\}$
    5. DP-Swap-Room N with 3 different cardinalities $N \in \{1, 2, 3\}$
- Time limit: 1 hour for each

Comparison of LNPS configurations on ITC-2007

Instance	*clingo*		*teaspoon-lnps*										
	bb	usc	Random N 2%	Random N 4%	Random N 6%	Day-Period	Day-Room	Swap-Room N 8%	Swap-Room N 10%	Swap-Room N 12%	DP-Swap-Room N 1	DP-Swap-Room N 2	DP-Swap-Room N 3
comp01	115	283	13	**11**	13	**11**	**11**	**11**	**11**	**11**	**11**	**11**	**11**
comp02	989	331	269	249	196	187	244	**178**	230	218	221	213	216
comp03	791	302	207	207	189	189	194	172	175	194	**165**	168	174
comp04	66	***49**	***49**	***49**	***49**	***49**	***49**	***49**	***49**	***49**	***49**	***49**	***49**
comp05	2662	1940	929	964	1011	**753**	863	1016	803	839	1040	936	978
comp06	777	1025	229	218	221	184	195	185	174	207	166	**156**	213
comp07	924	1149	196	197	342	163	172	**144**	243	251	203	233	181
comp08	332	***55**	***55**	***55**	***55**	***55**	***55**	***55**	***55**	***55**	***55**	***55**	***55**
comp09	563	254	183	175	174	165	161	173	169	160	168	179	**156**
comp10	821	1229	196	179	195	170	153	**127**	160	147	147	180	167
comp11	287	***0**	***0**	***0**	***0**	***0**	***0**	***0**	***0**	***0**	***0**	***0**	***0**
comp12	2626	1246	656	649	657	664	677	637	624	642	638	644	**622**
comp13	652	301	196	187	178	181	196	165	161	**160**	164	168	166
comp14	746	***67**	158	165	168	169	196	122	119	111	130	113	117
comp15	820	607	206	212	211	**194**	226	218	223	228	223	224	204
comp16	944	1090	191	199	244	209	217	**154**	199	177	173	200	173
comp17	979	412	216	218	239	234	238	193	**174**	207	194	178	214
comp18	503	340	158	153	158	152	158	146	149	144	148	151	**143**
comp19	890	919	186	198	209	191	205	182	182	189	194	188	**173**
comp20	3304	1386	362	391	996	403	361	400	**359**	402	393	451	544
comp21	891	310	255	268	281	238	247	193	197	179	215	**166**	183
#best bounds	0	4	3	4	3	6	4	**8**	6	5	5	6	**8**

- Swap-Room $N = 8$ and DP-Swap-Room $N = 3$ found the most number of best solutions.

Comparison with other approaches 1/3

Instance	Best known bound (♯)	*clingo*		*teaspoon-lnps*	
		Obtained bound	Rate to ♯ (%)	Obtained bound	Rate to ♯ (%)
comp01	11	115	+945	11	**0**
comp02	130	331	+154	178	**+36**
comp03	142	302	+112	165	**+16**
comp04	49	49	**0**	49	**0**
comp05	570	1940	+240	753	**+32**
comp06	85	777	+814	156	**+83**
comp07	42	924	+2100	144	**+242**
comp08	55	55	**0**	55	**0**
comp09	150	254	+69	156	**+4**
comp10	72	821	+1040	127	**+76**
comp11	0	0	**0**	0	**0**
comp12	483	1246	+157	622	**+28**
comp13	147	301	+104	160	**+8**
comp14	67	67	**0**	111	+65
comp15	176	607	+244	194	**+10**
comp16	96	944	+883	154	**+60**
comp17	155	412	+165	174	**+12**
comp18	137	340	+148	143	**+4**
comp19	125	890	+612	173	**+38**
comp20	124	1386	+1017	359	**+189**
comp21	151	310	+105	166	**+9**

- The previous best known bounds (♯) have been obtained by metaheuristic algorithms, ILP, SAT/MaxSAT, etc.
- We calculate $\frac{\text{obtained bound} - \sharp}{\sharp}$ as the rate of distance to the best known bounds.

Comparison with other approaches 2/3

Instance	Best known bound (♯)	*clingo*		*teaspoon-lnps*	
		Obtained bound	Rate to ♯ (%)	Obtained bound	Rate to ♯ (%)
DDS1	1831	6536	+256	3557	**+94**
DDS2	64	398	+521	70	**+9**
DDS3	22	22	**0**	22	**0**
DDS4	96	3123	+3153	1732	**+1704**
DDS5	76	76	**0**	76	**0**
DDS6	96	849	+784	167	**+73**
DDS7	52	645	+1140	56	**+7**
EA01	196	807	+311	207	**+5**
EA02	128	990	+673	113	**-11**
EA03	90	2555	+2738	660	**+633**
EA04	18	52	**+188**	311	+1627
EA05	14	19	+35	14	**0**
EA06	99	543	+448	152	**+53**
EA07	205	2831	+1280	1142	**+457**
EA08	40	276	+590	40	**0**
EA09	48	48	**0**	48	**0**
EA10	93	444	+377	362	**+289**
EA11	40	211	+427	40	**0**
EA12	27	234	+766	27	**0**

- *teaspoon-lnps* succeeds in finding an **improved bound** of EA02.

Comparison with other approaches 3/3

Instance	Best known bound (♯)	*clingo*		*teaspoon-lnps*	
		Obtained bound	Rate to ♯ (%)	Obtained bound	Rate to ♯ (%)
erlangen2011_2	12353	17035	+37	15480	**+25**
erlangen2012_1	28236	25061	**-11**	25763	**-8**
erlangen2012_2	37103	34360	**-7**	35220	**-5**
erlangen2013_1	28997	28302	**-2**	25339	**-12**
erlangen2013_2	30533	29140	**-4**	28870	**-5**
erlangen2014_1	28655	24510	**-14**	23926	**-16**
test1	232	607	+161	232	**0**
test2	20	20	**0**	20	**0**
test3	68	117	+72	76	**+11**
test4	166	260	+56	144	**-13**
toy	0	0	**0**	0	**0**
Udine1	138	953	+590	226	**+63**
Udine2	81	402	+396	86	**+6**
Udine3	37	333	+800	71	**+91**
Udine4	106	106	**0**	106	**0**
Udine5	47	518	+1002	46	**-2**
Udine6	36	285	+691	38	**+5**
Udine7	64	131	+104	64	**0**
Udine8	88	606	+588	107	**+21**
Udine9	56	163	+191	67	**+19**
UUMCAS_A131	19699	28688	+45	25698	**+30**
Average rate to ♯			+447		**+99**

- *teaspoon-lnps* succeeds in finding **improved bounds** of 5 erlangen instances, test4, and Udine5.
- *teaspoon-lnps* was able to significantly reduce the rate to +99% on average compared to +447% of *clingo*.

Related Work

- Besides LNPS [Sugimori+,'24], the use of adaptive LNS in ASP optimization has been explored [Eiter+,AAAI'22,KR'22,AIJ'24].
- ASP has been so far successfully applied in a wide variety of timetabling problems, such as
  - nurse scheduling problem [Alviano+,'22],
  - rehabilitation scheduling problem [Cardellini+,'24],
  - chemotherapy treatment scheduling problem [Dodaro+,'21],
  - personalized course schedule planning [Kahraman+,'19], and
  - shift design problem [Abseher+,'16].

By our results, it would be interesting to explore effective LNPS configurations for these problems.

Conclusion

We presented an approach to solve **CB-CTT** with **LNPS** based on ASP.

- *teaspoon-lnps* was able to reduce the ratio of previously best known bounds from +447% of *clingo* to **+99%**.
- We succeeded in finding improved bounds of **8** instances.

All source code is available from:
https://github.com/banbaralab/lpnmr2024 ▸ Web

Future work

1. Extending our approach to the most recent ITC-2019
2. Developing effective LNPS configurations for other timetabling problems, such as nurse scheduling problem